



**FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI**

**KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY**

**Semestrální práce z předmětu
Základy počítačové grafiky**

Krajinka

Jméno: Tomáš Svoboda
Datum odevzdání: 16. května 2026

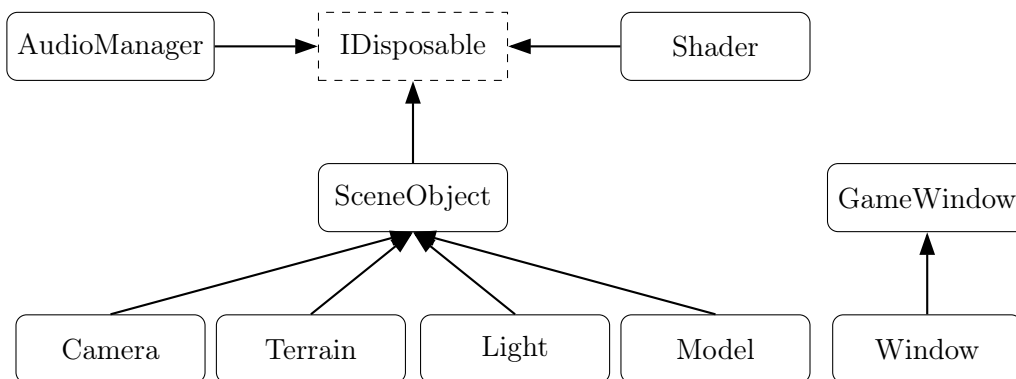
Obsah

1 Úvod	2
2 Architektura	2
3 Ovládání	3
3.1 Pohyb	3
3.2 Výběr mapy	3
4 Terén	3
4.1 Vstupní mapa	3
4.2 Výška terénu	3
4.3 Stavba meshe	3
4.4 Výška v daném bodě	4
4.5 Povrchy a textury	4
5 Kolize a pohyb pozorovatele	4
5.1 Přichycení k terénu	4
5.2 Skok a gravitace	4
5.3 Horizontální pohyb	4
5.4 Pohyb ve vodě	5
6 FPS kamera	5
7 Shader pipeline	5
7.1 Základní shader (basic.vert / basic.frag)	5
7.2 Shader pro vodu (basic.vert / water.frag)	5
7.3 Shader pro oblohu (sky.vert / sky.frag)	5
8 Objekty ve scéně	6
8.1 Implementace kolizního systému	6
8.2 Květiny	6
9 Zvukové efekty	6
10 Čas a FPS	7
11 Implementovaná rozšíření	7
11.1 Terén a povrchy	7
11.2 Atmosféra a osvětlení	7
11.3 Objekty a interakce	7
11.4 Zvukové efekty	7
12 Použité zdroje třetích stran	8
13 Využití nástrojů umělé inteligence	8

Úvod

Aplikace Krajinka je jednoduchý 3D renderer krajiny implementovaný v C# s využitím OpenTK (OpenGL 3.3 Core Profile) a .NET 8. Umožňuje procházet generovaný terén z pohledu první osoby, přičemž terén, rozmístění objektů a barevnost krajiny jsou načteny z RGBA PNG mapy.

Architektura



Aplikace je rozdělena do těchto hlavních tříd:

Třída	Funkce
Program	Vstupní bod, nastavení parametrů, výběr mapy
Window	Hlavní smyčka, zpracování vstupu, vykreslování
Terrain	Načtení PNG mapy, stavba meshe, dotaz na výšku
Camera	FPS kamera, ovládání pohybu, fyzika (gravitace, skok)
Model	Načtení a vykreslení OBJ modelu
ObjLoader	Parsování OBJ/MTL, výpočet normál
Shader	Kompilace GLSL shaderů, cache uniformů
SceneObject	Základní transformace a modelová matice
CollisionSystem	Detekce kolizí s objekty, hitboxy
AudioManager	Přehrávání zvukových efektů

Ovládání

Pohyb

Vstup	Akce
W / S / A / D	Pohyb vpřed / vzad / vlevo / vpravo
Myš	Otáčení kamery (yaw a pitch)
Mezerník	Skok
F11	Přepnutí fullscreen / okenní režim
Escape	Ukončení aplikace

Výběr mapy

Při spuštění aplikace se prohledá adresář `Data/maps/` a nalezené PNG soubory se vypíší do konzole s číselnými indexy. Uživatel zadá číslo požadované mapy a stiskne Enter. Pokud je v adresáři pouze jedna mapa, výběr se přeskočí a mapa se načte automaticky.

Terén

Vstupní mapa

Terén se generuje z RGBA PNG mapy načtené pomocí knihovny SkiaSharp, barevné kanály mají tento význam:

Kanál	Rozsah	Použití
R	0–255	Dolní bajt výšky
G	0–255	Kód objektu: 0 = nic, 1 = strom, 2 = kámen
B	0–255	Horní bajt výšky (rozšíření B+R na 16 bitů)
A	0–255	Informace o vodě (0 nebo 128 = voda)

Výška terénu

Výška se kóduje do dvou bajtů jako $h = B \times 256 + R$, čímž vzniká 16bitový rozsah (0–65535). Převod na světové jednotky: výška v metrech = $h \times 0.05$, protože 1 jednotka = 5 cm (`HeightScale = 0.05`).

Stavba meshe

Vzorky jsou rozmístěny na pravidelné mřížce s rozestupem `SampleSpacing = 0.5m`. Každý čtverec mřížky tvoří dva trojúhelníky. Normála je v každém bodě určena rozdílem výšek okolních vzorků.

Výška v daném bodě

Pro plynulý pohyb po terénu se výška v libovolném světovém bodě (x, z) určuje bilineární interpolací čtyř okolních vzorků mřížky.

Povrchy a textury

Terén není jednobarevný, ale na základě výšky, sklonu povrchu a blízkosti vody se automaticky přiřazuje jeden z těchto typů povrchu: tráva, skála, hlína nebo voda. Typ se používá pro výběr a prolínání vhodné textury.

Kolize a pohyb pozorovatele

Přichycení k terénu

Výška oka pozorovatele se každý snímek nastavuje na $h_{\text{ter}}(x, z) + 1.8 \text{ m}$ (`EyeHeight = 1.8`), kde výška 1,8 m odpovídá průměrné výšce očí dospělého člověka.

Skok a gravitace

Vertikální pohyb je simulován Eulerovou integrací. Vzhledem k tomu, že je měřítko světa nastaveno na **1 jednotka = 1 metr**, odpovídají konstanty v kódu přímo fyzikálním tabulkovým hodnotám:

Konstanta	Hodnota	
Gravity	9.81 m/s ²	Standardní zemská gravitace
JumpSpeed	3.70 m/s	Počáteční rychlost výskoku
GroundSnapEpsilon	8 cm	Tolerance dotyku s terénem

Horizontální pohyb

Rychlost pohybu `MovementSpeed` vychází z referenční rychlosti hráče ve hře Doom (1993) bez stisknuté klávesy Shift:

- **Původní rychlost v Doomu:** 291.66 mapových jednotek/s.
- **Převod na metry:** Při standardním poměru 32 jednotek = 1 metr je výsledná rychlost:

$$v = \frac{291.66}{32} \doteq 9.11 \text{ m/s}$$

Vstupní vektor směru je vždy normalizován, takže diagonální pohyb nezvyšuje výslednou rychlost. Pozice kamery je na obou horizontálních osách striktně oříznuta na hranice terénu.

Pohyb ve vodě

Při pohybu ve vodě je horizontální rychlost snížena na 60 % (`WaterSpeedMultiplier = 0.6`) normální rychlosti. Při vstupu do vody se přehrávají odpovídající zvukové efekty.

FPS kamera

Orientace kamery je definována úhly yaw a pitch. Pitch je omezen na $\langle -89^\circ, +89^\circ \rangle$, čímž se zabráňuje přetočení přes zenit. Směrový vektor `Front` se přepočítává ze sférických souřadnic. Pohledová matice se sestavuje přes `Matrix4.LookAt`, projekce je perspektivní (FOV 45° , near 0.1 m, far 1000 m).

Shader pipeline

Scéna se renderuje pomocí několika shaderů:

Základní shader (`basic.vert` / `basic.frag`)

Vertex shader transformuje vrcholy do světových souřadnic a předává normály a UV souřadnice dále. Fragment shader počítá osvětlení pomocí Lambertova difuzního modelu doplněného o konstantní ambientní světlo (0,15), aby i neosvětlená místa zůstala viditelná.

Pro terén shader vzorkuje `surfaceTypeMap` a podle její hodnoty plynule prolíná mezi texturami trávy, skály a hlíny pomocí `smoothstep`. Modely objektů používají vlastní texturu načtenou z MTL souboru.

Shader pro vodu (`basic.vert` / `water.frag`)

Hladina vody je renderována jako samostatná mesh se vlastním VAO/VBO/IBO, přičemž její vrcholy leží na pevné výšce `WaterSurfaceLevel = 1.08` m. Výjimkou jsou oblasti řek ve vyšší nadmořské výšce, kde hladina kopíruje výšku terénu. Animace vody je zajištěna cyklickým přepínáním mezi `WaterFrameCount = 40` snímků textury, přičemž každý snímek trvá `WaterFrameDuration = 0.08` s.

Voda sdílí vertex shader se základním shaderem. Fragment shader se však liší: kombinuje Lambertův difuzní model s Phongovým zrcadlovým odleskem (exponent 64) pro efekt lesklé vodní hladiny.

Shader pro oblohu (`sky.vert` / `sky.frag`)

Obloha se vykresluje jako jediný fullscreen trojúhelník bez geometrie, vertex shader vygeneruje tři vrcholy přímo z `gl_VertexID` a rekonstruuje směr pohledu do světových souřadnic pomocí inverzních matic projekce a pohledu.

Fragment shader kombinuje dva zdroje: vypočítaný přechod barev oblohy a cubemap texturu skyboxu. Vypočítaná část interpoluje mezi denními a nočními barvami na základě výšky slunce. Cubemap se prolíná s vypočítanou oblohou pouze poblíž horizontu, kde vyhlazuje přechod mezi nebem a terénem, výše se zobrazuje čistě vypočítané nebe.

Slunce je vykresleno jako disk s měkkým přechodem (**smoothstep**) podle skalárního součinu směru pohledu a směru slunce. Barva slunce se mění od oranžové při západu po bílou ve výšce.

Objekty ve scéně

Stromy (kód 1) a kameny (kód 2) jsou načteny a umístěny do mapy podle zeleného kanálu. Stromy nemohou být umístěny do vody, pokud je povrch skalnatý, jsou nahrazeny keřem. Každý objekt dostane náhodné měřítko (stromy 0.8–1.2, kameny 0.30–0.50, keře 0.015–0.027) a náhodnou rotaci kolem osy Y, aby scéna nevypadala monotónně. OBJ loader podporuje materiály z MTL souboru.

Implementace kolizního systému

Veškerá kolizní logika je zapouzdřena ve třídě **CollisionSystem**. Hitboxy objektů jsou implementovány jako zarovnané obdélníky v rovině XZ (AABB), což je dostatečně přesné pro účely této aplikace a zároveň výpočetně nenáročné.

Aby se při každém snímku neprohledával celý seznam objektů, jsou instance objektů uloženy do prostorové mřížky rozdělené podle vzorkovací mřížky terénu. Do kolizních testů se předávají pouze objekty z okolních buněk v poloměru 3 buněk (1,5 m) kolem predikované pozice hráče.

Před každým pohybem kamery se prediktivně vypočítá budoucí pozice hráče a ověří se, zda by v daném místě došlo k průniku s některým hitboxem. Pokud ano, pohyb je zablokován.

Kameny jsou speciálním případem: systém rozlišuje, zda se chodidla hráče nacházejí na úrovni vršku kamene (s tolerancí **RockTopStepEpsilon** = 8 cm), nebo zda hráč naráží do jeho boku. V prvním případě je pohyb povolen a hráč se po kameni pohybuje.

Květiny

Květiny se generují na vhodných, volných místech terénu. Hráč s nimi může interagovat, při fyzickém průchodu modelem květiny dojde k jejímu „zašlápnutí“ a odstranění ze scény. Při této interakci se přehraje zvukový efekt.

Zvukové efekty

Aplikace je vybavena jednoduchým audio systémem. Hra reaguje na akce hráče příslušnými zvukovými efekty:

- Zvuk kroků při pohybu po terénu (interval 0,9 s).
- Efekt žbluňknutí při skoku do vody.
- Specifický zvuk při zašlápnutí květiny.

Zvuky jsou přehrávány asynchronně bez zpoždění hlavní smyčky.

Čas a FPS

Veškerý pohyb je řízen pomocí `deltaTime`, takže je zcela nezávislý na snímkové frekvenci. FPS čítač v titulku okna zobrazuje klouzavý počet snímků za poslední 1 sekundu.

Implementovaná rozšíření

Projekt byl oproti základnímu zadání rozšířen v několika oblastech:

Terén a povrchy

- Automatické určování typu povrchu (tráva, skála, hlína, voda) podle výšky, sklonu a blízkosti vody.
- Prolínání textur mezi jednotlivými typy povrchů.
- Samostatně renderovaná animovaná hladina vody.

Atmosféra a osvětlení

- Denní a noční cyklus se změnou směru, intenzity a barvy globálního osvětlení (slunce).

Objekty a interakce

- Kolizní detekce s objekty pomocí hitboxů.
- Možnost stoupnout na vrch kamene.
- Generování květin na volných místech.
- Interakce se květinami (zašlápnutí a odstranění).

Zvukové efekty

- Zvukové efekty pro kroky, vstup do vody a interakce s květinami.

Použité zdroje třetích stran

Pro tvorbu vizuální a zvukové stránky aplikace byly využity následující volně dostupné prostředky:

- **Skybox:** Autor *elyvisions* (nahrál Calinou), dostupné na portálu OpenGameArt.
- **Model kamene:** Low-poly model (ID: 4631), dostupné na portálu Free3D.
- **Model růže (Rose flower):** Autor *SCANIMATE.IO* na platformě Sketchfab, licencováno pod Creative Commons Attribution (CC BY 4.0).
- **Model stromu:** Autor *storque12*, dostupné na portálu Free3D.
- **Ostatní 3D modely:** Další doplňkové modely byly získány z veřejných databází.
- **Textura vody:** Animovaná textura vody (128px), autor *YCbCr*, OpenGameArt.
- **Textura trávy:** Autor *p0ss*, OpenGameArt.
- **Textura hlíny (Mud):** Autor *qubodup*, OpenGameArt.
- **Textura skály (Rock):** Autor *Mysteryem* (nahrál qubodup), OpenGameArt.
- **Zvukové efekty:** Kolekce bezplatných zvukových stop z portálu *Mixkit.co*.

Využití nástrojů umělé inteligence

Při vývoji tohoto projektu byly využity nástroje generativní umělé inteligence v následujícím rozsahu:

- **Generování dokumentace:** XML komentáře ve zdrojovém kódu byly vygenerovány s pomocí umělé inteligence.
- **Asistence při psaní kódu:** Při programování byl aktivně využíván AI našepťávač integrovaný ve vývojovém prostředí Visual Studio.
- **Získávání informací:** Jazykový model Gemini (Google) byl využit k dohledávání informací, pochopení konceptů a řešení problémů namísto klasického manuálního prohledávání oficiálních dokumentací.

Souhlasím s vystavením této semestrální práce na stránkách katedry informatiky a výpočetní techniky a jejímu využití pro prezentaci pracoviště.